Section 3.4: Backpropagation

Computing Gradients with the Chain Rule

AI Class — Lecture Notes [Student Copy]

---

1 The Goal

From Section 3.2 (Gradient Descent), we know that to update a weight w we need its gradient:

$$w \longleftarrow w - \eta \frac{\partial J}{\partial w}.$$

From Section 3.3 (Forward Pass) we computed a prediction $\hat{y} = 0.637$ and a loss $J = 0.132$ for the input $(x_1, x_2) = (0, 1)$.

Now we need $\partial J/\partial w$ for every weight in the network. The difficulty is that the loss depends on a weight only *indirectly*: $w \to z \to h \to \cdots \to \hat{y} \to J$. Each arrow is a function, and we need to differentiate through all of them. The tool that makes this systematic is the **chain rule**.

2 The Chain Rule

If y depends on u, and u depends on x, then:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

This extends to any chain of functions. For three nested functions $y = f(g(h(x)))$, let $v = h(x)$ and $u = g(v)$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}.$$

Example 2.1. Let $y = (2x + 1)^2$. Write $u = 2x + 1$ so that $y = u^2$:

$$\frac{dy}{dx} = \underbrace{\frac{dy}{du}}_{2u} \cdot \underbrace{\frac{du}{dx}}_{2} = 2(2x + 1) \cdot 2 = \underline{\qquad\qquad\qquad\qquad}.$$

Backpropagation is simply the chain rule applied *backwards* through the network — one layer at a time. We start from $\partial J/\partial \hat{y}$ (easy to compute) and work leftward, accumulating the product at each layer.

3 What We Know from Section 3.3

We carry forward every value computed in the forward pass:

Symbol	Meaning	Value
$x_1,\ x_2$	inputs	$0,\ \ 1$
$z_1^{(1)},\ z_2^{(1)}$	hidden pre-activations	$0.0,\ \ 0.5$
$h_1,\ h_2$	hidden activations	$0.500,\ \ 0.622$
$z^{(2)}$	output pre-activation	0.561
$\hat{y}$	prediction	0.637
y	target	1
$J = (y - \hat{y})^2$	loss	0.132

We also need the sigmoid derivatives $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ at the values we computed:

Location	z	$\sigma'(z)$
$z_1^{(1)} = 0.0$	$\rightarrow$	$0.500 \times 0.500 = 0.250$
$z_2^{(1)} = 0.5$	$\rightarrow$	$0.622 \times 0.378 \approx 0.235$
$z^{(2)} \approx 0.561$	$\rightarrow$	$0.637 \times 0.363 \approx 0.231$

4 Backward Pass: Output Layer

We work right to left through the computation graph:

$$J \xleftarrow{\partial J/\partial \hat{y}} \hat{y} \xleftarrow{\partial \hat{y}/\partial z^{(2)}} z^{(2)} \xleftarrow{\partial z^{(2)}/\partial w^{(2)}} w^{(2)}.$$

Step 1 Gradient of the loss with respect to $\hat{y}$

$$J = (y - \hat{y})^2 \implies \frac{\partial J}{\partial \hat{y}} = 2(\hat{y} - y) = 2(0.637 - 1) = \underline{\qquad\qquad}$$

The negative sign means: increasing $\hat{y}$ would *decrease* J. Since $\hat{y} < y$, the network is predicting too low.

Step 2 Gradient through the output sigmoid

$$\hat{y} = \sigma(z^{(2)}) \implies \frac{\partial \hat{y}}{\partial z^{(2)}} = \sigma'(z^{(2)}) = \hat{y}(1 - \hat{y}) = 0.637 \times 0.363 \approx \underline{\qquad\qquad}$$

By the chain rule, the gradient arriving at $z^{(2)}$ is:

$$\frac{\partial J}{\partial z^{(2)}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}} = (-0.726)(0.231) \approx \underline{\qquad\qquad}$$

We will call this quantity $\delta^{(2)} = -0.168$. It is the **error signal** at the output neuron: how much the loss changes per unit change in $z^{(2)}$.

Step 3 Gradients for the output weights

Because $z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 + b^{(2)}$, we have:

$$\frac{\partial z^{(2)}}{\partial w_1^{(2)}} = h_1, \qquad \frac{\partial z^{(2)}}{\partial w_2^{(2)}} = h_2, \qquad \frac{\partial z^{(2)}}{\partial b^{(2)}} = 1.$$

Applying the chain rule one more time:

$$\frac{\partial J}{\partial w_1^{(2)}} = \delta^{(2)} \cdot h_1 = (-0.168)(0.500) = \_\_\_\_\_$$

$$\frac{\partial J}{\partial w_2^{(2)}} = \delta^{(2)} \cdot h_2 = (-0.168)(0.622) \approx \_\_\_\_\_$$

$$\frac{\partial J}{\partial b^{(2)}} = \delta^{(2)} \cdot 1 = \_\_\_\_\_$$

Pattern observed. For any output weight $w_i^{(2)}$, the gradient is:

$$\frac{\partial J}{\partial w_i^{(2)}} = \delta^{(2)} \cdot h_i \qquad \text{(error signal} \times \text{incoming activation).}$$

This pattern repeats at every layer.

5 Backward Pass: Hidden Layer

To update $\mathbf{W}^{(1)}$ we need $\partial J / \partial w_{ij}^{(1)}$. The chain from $w_{ij}^{(1)}$ to J goes:

$$w_{ij}^{(1)} \;\to\; z_i^{(1)} \;\to\; h_i \;\to\; z^{(2)} \;\to\; \hat{y} \;\to\; J.$$

We already know $\delta^{(2)} = \partial J / \partial z^{(2)} = -0.168$. The chain rule gives the error signal at hidden unit i:

Error signal at hidden unit i:

$$\delta_i^{(1)} = \frac{\partial J}{\partial z_i^{(1)}} = \delta^{(2)} \cdot w_i^{(2)} \cdot \sigma'\left(z_i^{(1)}\right).$$

Three factors multiplied together:

1. $\delta^{(2)}$: error signal from the layer ahead.
2. $w_i^{(2)}$: how strongly h_i connects to the output.
3. $\sigma'(z_i^{(1)})$: how sensitive h_i is to its own pre-activation.

Once $\delta_i^{(1)}$ is known, the weight gradients follow the same rule as before:

$$\frac{\partial J}{\partial w_{ij}^{(1)}} = \delta_i^{(1)} \cdot x_j, \qquad \frac{\partial J}{\partial b_i^{(1)}} = \delta_i^{(1)}.$$

5.1 Applying the Pattern to h_1

$$\delta_1^{(1)} = \delta^{(2)} \cdot w_1^{(2)} \cdot \sigma'(z_1^{(1)}) = (-0.168)(0.5)(0.250) = \_\_\_\_\_$$

The weight gradients for row 1 of $\mathbf{W}^{(1)}$:

$$\frac{\partial J}{\partial w_{11}^{(1)}} = \delta_1^{(1)} \cdot x_1 = (-0.021)(0) = \_\_\_\_\_$$

$$\frac{\partial J}{\partial w_{12}^{(1)}} = \delta_1^{(1)} \cdot x_2 = (-0.021)(1) = \_\_\_\_\_$$

$$\frac{\partial J}{\partial b_1^{(1)}} = \delta_1^{(1)} = \_\_\_\_\_$$

$\partial J/\partial w_{11}^{(1)} = 0$ because $x_1 = 0$: this input was zero, so the weight connecting it had no effect on the prediction — and therefore no effect on the loss.

5.2 Applying the Pattern to h_2

$$\delta_2^{(1)} = \delta^{(2)} \cdot w_2^{(2)} \cdot \sigma'(z_2^{(1)}) = (-0.168)(0.5)(0.235) \approx \_\_\_\_\_$$

$$\frac{\partial J}{\partial w_{21}^{(1)}} = \delta_2^{(1)} \cdot x_1 = (-0.020)(0) = \_\_\_\_\_$$

$$\frac{\partial J}{\partial w_{22}^{(1)}} = \delta_2^{(1)} \cdot x_2 = (-0.020)(1) = \_\_\_\_\_$$

$$\frac{\partial J}{\partial b_2^{(1)}} = \delta_2^{(1)} = \_\_\_\_\_$$

6 Summary of All Gradients

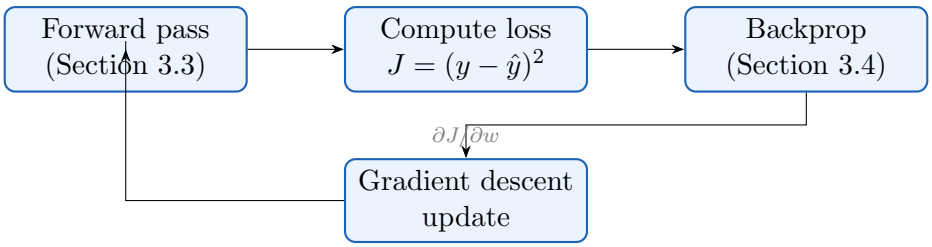
Weight	Gradient $\partial J/\partial w$	GD update ($\eta = 0.1$)
$w_1^{(2)}$	-0.084	$0.5 - 0.1(-0.084) = 0.508$
$w_2^{(2)}$	-0.105	$0.5 - 0.1(-0.105) = 0.511$
$b^{(2)}$	-0.168	$0.0 - 0.1(-0.168) = 0.017$
$w_{11}^{(1)}$	0.000	$0.5 - 0.1(0.000) = 0.500$
$w_{12}^{(1)}$	-0.021	$-0.5 - 0.1(-0.021) = -0.498$
$b_1^{(1)}$	-0.021	$0.5 - 0.1(-0.021) = 0.502$
$w_{21}^{(1)}$	0.000	$0.5 - 0.1(0.000) = 0.500$
$w_{22}^{(1)}$	-0.020	$0.5 - 0.1(-0.020) = 0.502$
$b_2^{(1)}$	-0.020	$0.0 - 0.1(-0.020) = 0.002$

All gradients are **negative** — every weight, when increased, would decrease the loss. So gradient descent moves every weight up slightly. After one update step the network would output a value slightly closer to 1 for this input.

One forward–backward pass updates weights for one input. In practice we cycle through all four XOR inputs (or use mini-batches), accumulate gradients, and take one update step per batch. After hundreds or thousands of such cycles the network converges and correctly classifies all four XOR inputs.

7 The Big Picture

The full training loop is now complete in principle:



Coming next — Section 3.5: Adam. Plain gradient descent works, but it can be slow or sensitive to the learning rate. Adam is a smarter update rule that adapts the step size for each weight automatically. We will see how it improves on plain gradient descent with only a small change to the update formula.